

P3612R1

Harmonize proxy-reference operations (LWG 3638 and 4187)

Published Proposal, 2025-11-06

Author:

[Arthur O'Dwyer](#)

Audience:

LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposed resolutions for two interrelated LWG issues. LWG 3638 adds ADL swap to `vector<bool>::reference`. LWG 4187 adds const-assignability to `bitset<N>::reference`. These two types' value-semantic operations should not gratuitously diverge. Use the same signatures and specification techniques for both.

Table of Contents

- 1 Changelog
- 2 Background
- 3 Don't mandate triviality
- 4 Proposed wording
 - 4.1 [template.bitset.general]
 - 4.2 [vector.bool]
 - 4.3 [depr.vector.bool.swap]

References

Informative References

§ 1. Changelog

- R0 (pre-Hagenberg 2025): Initial revision.
- R1 (during Kona 2025): Two editorial corrections. Approved by LWG for C++26.

§ 2. Background

From scratch, the simplest proxy-reference type for `bool` would look like this:

```
struct Reference {
    bool *p_;
    Reference(bool& r) : p_(&r) {}
    Reference(const Reference&) = default;
    Reference operator=(Reference rhs) const {
        *p_ = *rhs.p_;
        return *this;
    }
    operator bool() const {
        return *p_;
    }
    Reference operator=(bool b) const {
        *p_ = b;
        return *this;
    }
    friend void swap(Reference lhs, Reference rhs) {
        std::swap(*lhs.p_, *rhs.p_);
    }
};
```

1. Let `r` be a variable of type `Reference`. We need `operator=(bool)` so that `r = true` will compile. But we also need `operator=(Reference)` so that the compiler will not generate us a defaulted copy-assignment operator. Both `operator=`s are const-qualified following [P2321]'s guidance (§5.3); see also "[Field-testing -Wassign-to-class-rvalue](#)".

2. Now, C++20's `vector<bool>::reference` already had a non-const-qualified `operator=(bool)`. It would have broken ABI if [P2321] had simply *added* `const` to that member function. Therefore P2321 added a whole new const-qualified overload of `operator=(bool)` alongside the pre-existing non-const-qualified `operator=(bool)`. We wouldn't do that from scratch, but we do it for backward compatibility.

3. P2321 also established the precedent that `operator=(bool)` and `operator=(bool) const` return lvalue references to `Reference`, rather than prvalues of type `Reference`. We wouldn't do that from scratch either, but we do it now because of P2321.

4. We don't need an `operator=(const Reference&) const` because `Reference` is already implicitly convertible to `bool`. If `cr` is of type `const Reference`, then `cr = cr2` will end up calling `cr.operator=(bool(cr2))`, which is fine. (Again, [P2321] has already established this pattern.)

5. We must provide ADL `swap`. The generic `std::swap` is inappropriate for two reasons:

- We need `swap(v[1], v[2])` to work even when the expression `v[1]` is a prvalue `Reference`. The generic `std::swap(T&, T&)` doesn't accept prvalue arguments.
- The generic algorithm `Reference t = r1; r1 = r2; r2 = t;` fails to swap the values of the referents of `r1` and `r2`.

If this `Reference` is the `iter_reference_t` of some iterator type, then that iterator type might do well also to provide an `iter_swap`; but that doesn't affect the rationale above; we must invariably provide an ADL `swap`.

6. Consider `swap(r, b)`, where `b` is an lvalue of type `bool`. If `bool&` is implicitly convertible to `Reference`, this will happily use the single overload of ADL `swap` above. But in the STL, `vector<bool>::reference` and `bitset<N>::reference` are not implicitly convertible from `bool&`. (This is as it should be: you shouldn't be able to create one referring to an arbitrary `bool` of your own.) So, in order to make `swap(r, b)` and `swap(b, r)` compile, we'll need two additional overloads of `swap`. This matches the proposed resolution of [LWG3638].

The final product (omitting `constexpr` and `noexcept`) looks like this:

```
struct Reference {
    bool *p_;
    explicit Reference(bool* p) : p_(p) {} // exposition only
    Reference(const Reference&) = default;
    Reference& operator=(const Reference& rhs) { // C++98
        *p_ = *rhs.p_;
```

```

    return *this;
}
operator bool() const {
    return *p_;
}
Reference& operator=(bool b) { // C++98
    *p_ = b;
    return *this;
}
const Reference& operator=(bool b) const { // P2321
    *p_ = b;
    return *this;
}
friend void swap(Reference lhs, Reference rhs) {
    std::swap(*lhs.p_, *rhs.p_);
}
friend void swap(Reference lhs, bool& rhs) {
    std::swap(*lhs.p_, rhs);
}
friend void swap(bool& lhs, Reference rhs) {
    std::swap(lhs, *rhs.p_);
}
};

```

We propose to apply this pattern consistently in both `vector<bool>::reference` (resolving [LWG3638]) and `bitset<N>::reference` (resolving [LWG4187]).

§ 3. Don't mandate triviality

Today the copy constructors of `vector<bool>::reference` and `bitset::reference` are specified with `=default`; but because we don't specify their data members, this says nothing normative about the copy constructor's noexceptness and triviality. These explicitly defaulted declarations were added to C++20 by P0619, merely to avoid relying on [depr.impldec]. Prior to C++20 these types did not specify a copy constructor at all, which was interpreted as a request for implicitly defaulted copy and move constructors, which *still* said nothing normative about noexceptness and triviality.

Both copy constructors are trivial in practice, on all three vendors.

However, there is implementation divergence on the destructors. All vendors give `vector<bool>::reference` a trivial destructor; but only libc++ gives `bitset::reference` a

trivial destructor. `libstdc++` and Microsoft give `bitset::reference` a non-trivial user-provided destructor. This means that the two types have visibly different calling conventions ([Godbolt](#)). We cannot mandate a change here, because ABI.

	Trivial copy constructor?			Trivial destructor?		
	libstdc++	libc++	Microsoft	libstdc++	libc++	Microsoft
<code>vector<bool>::reference</code>	Yes	Yes	Yes	Yes	Yes	Yes
<code>bitset<N>::reference</code>	Yes	Yes	Yes	No	Yes	No

We conceivably *could* require triviality of any operation with "Yes"es all across its row above. But, since we can't get it for `bitset::reference`'s destructor; I don't want to introduce gratuitous differences in specification between `bitset::reference` and `vector<bool>::reference`; and it doesn't seem very important whether a proxy reference's *copy constructor* is normatively specified to be trivial — I don't think it's worth the bother to specify.

§ 4. Proposed wording

§ 4.1. [\[template.bitset.general\]](#)

DRAFTING NOTE: This resolves [\[LWG4187\]](#). We don't add a const-qualified overload of `reference::flip` because I don't think anyone cares about `flip`. We don't rearrange the members to match `[vector.bool]`'s order because that can be done later, editorially.

Modify [\[template.bitset.general\]](#) as follows:

```

namespace std {
    template<size_t N> class bitset {
    public:
        // bit reference
        class reference {
        public:
            constexpr reference(const reference& x) noexcept = default;
            constexpr ~reference();
            constexpr reference& operator=(bool x) noexcept;           // for b[i] = x;
            constexpr reference& operator=(const reference& x) noexcept; // for b[i] = b[j]
            constexpr const reference& operator=(bool x) const noexcept;
            constexpr bool operator~() const noexcept;                // flips the bit
            constexpr operator bool() const noexcept;                 // for x = b[i];
            constexpr reference& flip() noexcept;                      // for b[i].flip();
            friend constexpr void swap(reference x, reference y) noexcept;
            friend constexpr void swap(reference x, bool& y) noexcept;
            friend constexpr void swap(bool& x, reference y) noexcept;
        };

        [...]

    };

    // [bitset.hash], hash support
    template<class T> struct hash;
    template<size_t N> struct hash<bitset<N>>;
}

```

1. The class template `bitset<N>` describes an object that can store a sequence consisting of a fixed number of bits, `N`.
2. Each bit represents either the value zero (reset) or one (set). To *toggle* a bit is to change the value zero to one, or the value one to zero. Each bit has a non-negative position `pos`. When converting between an object of `class type` `bitset<N>` and a value of some integral type, bit position `pos` corresponds to the *bit value* `1 << pos`. The integral value corresponding to two or more bits is the sum of their bit values.

x. reference is a class that simulates a reference to a single bit in the sequence.

constexpr reference::reference(const reference& x) noexcept;

x. Effects: Initializes *this to refer to the same bit as x.

constexpr reference::~reference();

x. Effects: None.

constexpr reference& reference::operator=(bool x) noexcept;

constexpr reference& reference::operator=(const reference& x) noexcept;

constexpr const reference& reference::operator=(bool x) const noexcept;

x. Effects: Sets the bit referred to by `*this` if `bool(x)` is true, and clears it otherwise.

x. Returns: `*this`.

```
constexpr void swap(reference x, reference y) noexcept;  
constexpr void swap(reference x, bool& y) noexcept;  
constexpr void swap(bool& x, reference y) noexcept;
```

x. Effects: Exchanges the values denoted by `x` and `y` as if by:

```
bool b = x;  
x = y;  
y = b;
```

```
constexpr reference& reference::flip() noexcept;
```

x. Effects: Equivalent to `*this = !*this`.

x. Returns: `*this`.

3. The functions described in [template.bitset] can report three kinds of errors, each associated with a distinct exception [...]

§ 4.2. [vector.bool]

DRAFTING NOTE: This resolves [LWG3638].

Modify [vector.bool] as follows:

```

namespace std {
    template<class Allocator>
    class vector<bool, Allocator> {
    public:
        // types
        using value_type          = bool;
        using allocator_type      = Allocator;
        using pointer              = implementation-defined;
        using const_pointer       = implementation-defined;
        using const_reference     = bool;
        using size_type           = implementation-defined; // see [container.requiremen
        using difference_type     = implementation-defined; // see [container.requiremen
        using iterator             = implementation-defined; // see [container.requiremen
        using const_iterator      = implementation-defined; // see [container.requiremen
        using reverse_iterator    = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // bit reference
        class reference {
        public:
            constexpr reference(const reference&) noexcept = default;;
            constexpr ~reference();
            constexpr operator bool() const noexcept;
            constexpr reference& operator=(bool x) noexcept;
            constexpr reference& operator=(const reference& x) noexcept;
            constexpr const reference& operator=(bool x) const noexcept;
            constexpr void flip() noexcept;    // flips the bit
            friend constexpr void swap(reference x, reference y) noexcept;
            friend constexpr void swap(reference x, bool& y) noexcept;
            friend constexpr void swap(bool& x, reference y) noexcept;
        };
    };

```

[...]

```

        constexpr void swap(vector&)
            noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
                    allocator_traits<Allocator>::is_always_equal::value);
        static constexpr void swap(reference x, reference y) noexcept;
        constexpr void flip() noexcept;    // flips all bits
        constexpr void clear() noexcept;
    };
}

```

[...]

4. reference is a class that simulates **a reference to a single bit in the sequence**. ~~the behavior of references of a single bit in vector<bool>. The conversion function returns true when the bit is set, and false otherwise. The assignment operators set the bit when the argument is (convertible to) true and clear it otherwise. flip reverses the state of the bit.~~

```
constexpr reference::reference(const reference& x) noexcept;
```

x. Effects: Initializes `*this` to refer to the same bit as `x`.

```
constexpr reference::~~reference();
```

x. Effects: None.

```
constexpr reference& reference::operator=(bool x) noexcept;
```

```
constexpr reference& reference::operator=(const reference& x) noexcept;
```

```
constexpr const reference& reference::operator=(bool x) const noexcept;
```

x. Effects: Sets the bit referred to by `*this` when `bool(x)` is true, and clears it otherwise.

x. Returns: `*this`.

```
constexpr void reference::flip() noexcept;
```

x. Effects: Equivalent to `*this = !*this`.

```
constexpr void swap(reference x, reference y) noexcept;
```

```
constexpr void swap(reference x, bool& y) noexcept;
```

```
constexpr void swap(bool& x, reference y) noexcept;
```

x. Effects: Exchanges the values denoted by `x` and `y` as if by:

```
bool b = x;
```

```
x = y;
```

```
y = b;
```

```
constexpr void flip() noexcept;
```

5. *Effects:* Replaces each element in the container with its complement.

```
static constexpr void swap(reference x, reference y) noexcept;
```

~~6. *Effects:* Exchanges the contents of `x` and `y` as if by:~~

```
bool b = x;
```

```
x = y;
```

```
y = b;
```

§ 4.3. [depr.vector.bool.swap]

Create a new subclause [depr.vector.bool.swap] under [depr]:

D.? Deprecated `vector<bool, Allocator> swap` [depr.vector.bool.swap]

x. The following member is declared in addition to those members specified in [vector.bool]:

```
namespace std {  
    template<class Allocator> class vector<bool, Allocator> {  
    public:  
        static constexpr void swap(reference x, reference y) noexcept;  
    };  
}  
  
static constexpr void swap(reference x, reference y) noexcept;
```

x. *Effects*: Exchanges the values denoted by x and y as if by:

```
bool b = x;  
x = y;  
y = b;
```

§ References

§ Informative References

[LWG3638]

Jonathan Wakely. *`vector<bool>::swap(reference, reference)` is useless*. November 2021. URL: <https://cplusplus.github.io/LWG/issue3638>

[LWG4187]

Arthur O'Dwyer. *`bitset::reference` should be const-assignable*. December 2024. URL: <https://cplusplus.github.io/LWG/issue4187>

[P2321]

Tim Song. *`zip`*. June 2021. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2321r2.html>